

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Information security</i>
<i>Instructor</i>	<i>MS. Ambreen Gull</i>
<i>Assignment No.</i>	<i>03</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>Fa24-bse-039</i>	<i>M. Taha Niaz</i>

Task 1: Basic Blockchain Implementation

This implementation focuses on the "chain" aspect—specifically how cryptographic hashes create an immutable link between data blocks.

```
import hashlib
import time

class Block: 2 usages
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self): 2 usages
        # Combines block contents into a single string to hash
        block_string = f"{self.index}-{self.timestamp}-{self.data}-{self.previous_hash}"
        return hashlib.sha256(block_string.encode()).hexdigest()

class Blockchain: 1 usage
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self): 1 usage
        return Block(index=0, data="Genesis Block", previous_hash="0")

    def add_block(self, data): 3 usages
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)

    def display_chain(self): 1 usage
        for block in self.chain:
            print(f"Index: {block.index} | Hash: {block.hash[:20]}... | Prev: {block.previous_hash[:20]}...")
```

```
# --- Execution ---
my_coin = Blockchain()
my_coin.add_block("Sent 1.5 BTC to Alice")
my_coin.add_block("Sent 0.5 BTC to Bob")
my_coin.add_block("Sent 10 BTC to Charlie")

print("--- Initial Chain ---")
my_coin.display_chain()

print("\n--- Tampering with Block 1 ---")
my_coin.chain[1].data = "Sent 100 BTC to Hacker" # Modifying data
print(f"New Hash of Block 1: {my_coin.chain[1].calculate_hash()[:20]}...")
print(f"Stored Previous Hash in Block 2: {my_coin.chain[2].previous_hash[:20]}...")
```

Line-by-Line Explanation

1. **hashlib & time**: Used for the SHA-256 algorithm and generating unique timestamps.
2. **calculate_hash()**: This is the "digital fingerprint." It takes all the block's data and runs it through SHA-256. If a single character in data changes, the resulting hash changes entirely (the avalanche effect).
3. **previous_hash**: This is the glue. Each block stores the hash of the block before it.
4. **create_genesis_block()**: The first block in any chain. Since there is no predecessor, the previous_hash is manually set to "0".
5. **add_block()**: Grabs the hash of the current tip of the chain (self.chain[-1].hash) and assigns it as the previous_hash for the new block.

Security Analysis

- **Immutability**: Because each block references the previous hash, changing Block 1 changes its hash. Consequently, Block 2 (which stores the old hash) becomes invalid. To "hack" the chain, you would have to recalculate every subsequent block in the chain.
- **Weakness**: This implementation lacks **Consensus (Proof of Work)**. A modern computer could recalculate millions of hashes per second, making it easy to "re-write" history. Adding a "nonce" and a difficulty target would solve this.

Task 2: Elliptic Curve Cryptography (ECC) Basics

```
# Parameters for  $y^2 = x^3 + 2x + 2 \pmod{17}$ 
P = 17
A = 2
B = 2

def point_add(P1, P2, p, a): 4 usages
    if P1 is None: return P2
    if P2 is None: return P1
    x1, y1 = P1
    x2, y2 = P2

    if x1 == x2 and y1 != y2: return None # Point at infinity

    # Calculate Slope (m)
    if x1 == x2: # Point Doubling
        m = (3 * x1 ** 2 + a) * pow(2 * y1, -1, p)
    else: # Point Addition
        m = (y2 - y1) * pow(x2 - x1, -1, p)

    m %= p
    x3 = (m ** 2 - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p
    return (x3, y3)

def scalar_mult(k, point, p, a): 4 usages
    result = None
    addend = point
    while k:
        if k & 1: result = point_add(result, addend, p, a)
        addend = point_add(addend, addend, p, a)
```

```

        k >= 1
    return result

# --- ECC Key Gen & Simplified ElGamal ---
G = (5, 1) # Generator point on our curve
priv_key = 7
pub_key = scalar_mult(priv_key, G, P, A) # Public Key B = dG

# Encryption of message M (point on curve)
M = (9, 1) # Let's assume our message maps to this point
k = 3 # Ephemeral random key
C1 = scalar_mult(k, G, P, A)
C2 = point_add(M, scalar_mult(k, pub_key, P, A), P, A)

# Decryption: M = C2 - (priv_key * C1)
shared_secret = scalar_mult(priv_key, C1, P, A)
neg_shared_secret = (shared_secret[0], (-shared_secret[1]) % P)
decrypted_M = point_add(C2, neg_shared_secret, P, A)

print(f"Original Message Point: {M}")
print(f"Decrypted Message Point: {decrypted_M}")

```

Line-by-Line Explanation

1. Slope Calculation (m):

- For **Addition**, $m = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}$.
- For **Doubling**, $m = \frac{3x_1^2 + a}{2y_1} \pmod{p}$.

- pow(n, -1, p):** This calculates the modular multiplicative inverse, effectively performing "division" in a finite field.
- Scalar Multiplication:** This is the "trapdoor function." It's easy to compute $Q = kG$ (adding G to itself k times), but computationally impossible to find k if you only know Q and G .
- Encryption (ElGamal):** We produce a pair (C_1, C_2) . C_1 is the ephemeral public key, and C_2 is the message "masked" by the shared secret.
- Decryption:** We use the private key to unmask the shared secret from C_1 and subtract it from C_2 .

Security Analysis

- The Discrete Log Problem:** The security relies on the fact that while $P + P$ is easy, reversing the process to find how many times P was added to itself is very hard.
- Small Parameters:** Using $p=17$ is purely for education. In real systems (like Bitcoin's secp256k1), p is a 256-bit number, making brute-force attacks impossible with current technology.

- **Efficiency:** ECC provides the same security level as RSA but with much smaller key sizes (e.g., a 256-bit ECC key is roughly equivalent to a 3072-bit RSA key), making it ideal for mobile devices and blockchains.